PORTFOLIO

2025

엔진의 바닥까지 파고드는 개발자

오승찬

CONTACT

PHONE : 010 8004 7827 | MAIL : chanipm29@gmail.com

오승찬

www.notion.so/c7561b3293d544d4814d4b5d8597cb5f?source=copy_link

PORTFOLIO

INTRODUCTION

오승찬 1998.11.03 010-8004-7827 chanipm29@gmail.com

학력사항

2017.03~2025.02 한국외국어대학교

학과	학점	졸업 여부
정보통신공학과	3.27 / 4.5	졸업

경력사항

2023.03~2024.03 미소정보기술 인턴

CV 기술을 Unity 환경에 적용해 XR 콘텐츠의 상호작용성을 강화
대학 연구진과의 협업을 통해 기술 프로토타입 제작
관계자 대상 시연을 진행하고 납품을 위한 기술 검증

2024.10~2025.07 콘텐츠깎는청년 개발팀 근무

Unity 기반으로 하이퍼 캐주얼 모바일 게임을 빠르게 기획 및 출시
기획→개발→배포 주기를 2~3일 단위로 단축, 총 15개 이상의 프로토타입 제작
유저 유지율과 광고 수익을 중심으로 반복적인 개선 사이클을 수행
A/B 테스트를 통해 국가·연령별 플레이 패턴을 분석 -> UI/광고 빈도·세션 구조를 개선

교육사항

2025.07~2026.01 프로그래밍 기반 얼라인 프로젝트

교육기관	분야	수료여부
원티드랩	언리얼	수료예정

언리얼 엔진 핵심 기술 학습
AI 에이전트 트랙과 협업 언리얼 기반 콘텐츠 시뮬레이션 구현

어떤 역량을 가지고 있을까

| 스킬 및 역량

Languages

Proficient : C++, C#

Familiar : JavaScript, Python, SQL

Engines & Frameworks

Unreal Engine, Unity Engine, ASP.Net Core, .Net, Node.js

Core Competencies

Network: TCP/IP, Socket Programming, Protobuf, Async IO

Architecture: OOP, Component-Based Design, Data-Driven Design, FSM

Computer Science: Data Structure, Algorithm, OS (Thread/Process)

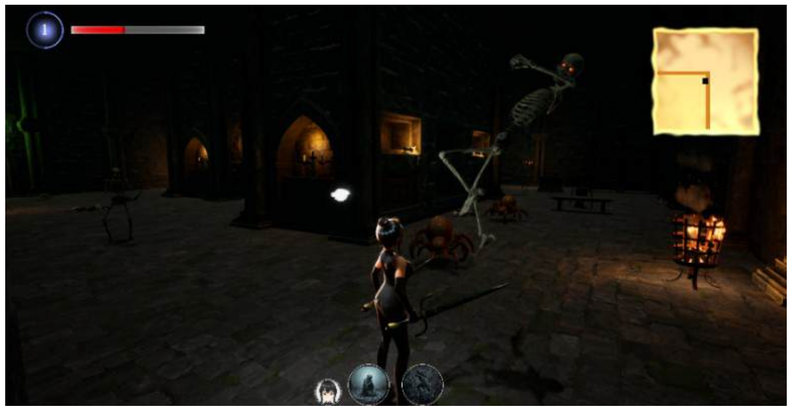
PORTFOLIO

INDEX

Memoria Lybrinth	**자체 전투 아키텍처 / 절차적 생성 알고리즘** GAS를 대체하는 경량화 시스템 설계와 MST 기반의 동적 던전 생성 구현
화재 대비 시뮬레이터	**고급 자료구조 활용 최적화 / 멀티 플레이어 관리** 언리얼 엔진의 멀티플레이 기능 활용과 자체적 자료구조를 통한 최적화
헬 다이버즈	**컴포넌트 기반 모듈형 설계** 모듈화 분업을 통해 효율적인 협업 구조 설계
타이니 서바이버	**비동기 소켓 프로그래밍 / 멀티스레딩** .Net 기반 C# TCP 소켓 서버 구현
협업 관리	**간트 차트 / 데이터 시트** 문서를 통해 서로의 일정을 공유, 데이터도 외부 문서를 통해 공유

Memoria Labyrinth

기간 : 25.11.04 ~ 26.01.07



AI 기반 멀티 플레이 로그라이크
생성형 AI가 만든 실시간 콘텐츠와 로그라이크의 복잡한 상호작용을
확장성 있게 구현한 RPG

역할

어빌리티 시스템 설계, 절차적 생성 알고리즘 구현
주요 데이터 설계

참여 인원

총 6명

영상 링크

https://drive.google.com/file/d/1AiHqwUxGb1vdTQdzAAdi3M0G3BQLZWo6/view?usp=drive_link

깃허브 링크

https://github.com/rouge-like/ProjectFP

주요 성과

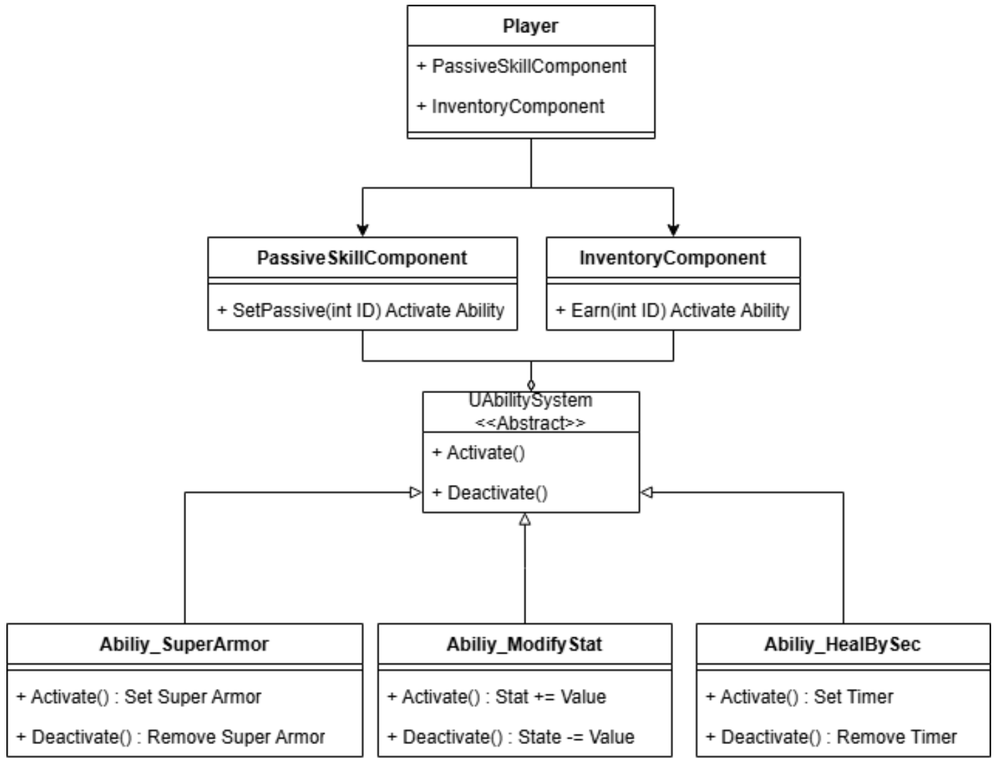
Poten Up 교육과정 최종프로젝트 최우수상 수상

Custom Lightweight Ability System 프로젝트에 최적화된 어빌리티 시스템 설계

GAS

문제 단순한 수치 변동을 위해 GAS는 과도한 객체 생성과 복제를 요구, 변동이 빈번한 로그라이크에서 이는 불필요한 부하를 유발함
기획 변경이 잦은 개발 초기 단계에, 직관적으로 제어가 어려움

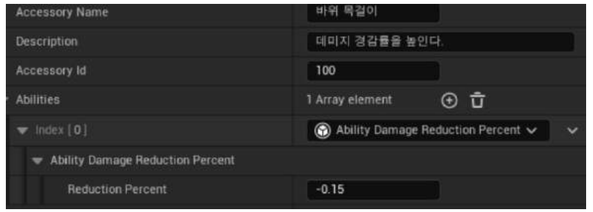
판단 GAS의 설계 철학을 계승하여 경량화한 자체 시스템 설계



계승 모든 액션, 스킬을 독립된 객체(UObject)로 캡슐화 하여 관리 / 전략 패턴
Actor가 아닌 Component가 생명주기 관리
ID와 데이터 테이블을 통해 로직과 데이터를 분리

개선 복잡한 AttributeSet 연산 대신 Player의 요소와 직접 통신하여 직관적으로 제어
Activate-Deactivate 명확한 대칭 구조로 상태 오염 원천 차단

[패시브 스킬 DataAsset]



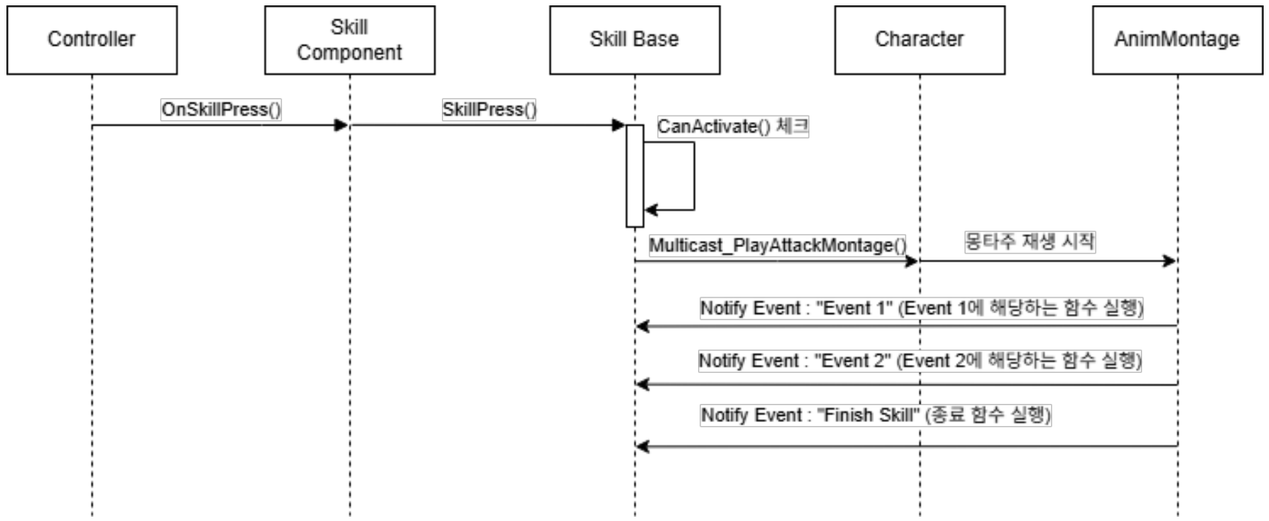
[아이템 DataAsset]

Skill System Architecture

몽타주와 노티파이를 활용한 소울라이크 액션 동기화

단순히 수치만 바꾸는 버프와 달리, 액션 스킬은 '특정 프레임'에 로직이 필요함

애니메이션이 실행 흐름을 주도하되, 실제 로직은 C++ 클래스가 담당하여 아트와 코드의 의존성 분리



```cpp
void USkillEventNotify::Notify(USkeletalMeshComponent* MeshComp, UAnimSequenceBase* Animation)
{
    // Owner -> SkillComponent -> CurrentActiveSkill 로 이벤트 전달
    if (auto* SkillComp = Player->FindComponentByClass<USkillComponent>())
    {
        SkillComp->GetSkill()->HandleSkillNotify(EventName);
    }
}

// [SkillBase.cpp] 전달받은 이벤트 처리
void USkillBase::OnSkillEvent(FName EventName)
{
    if (EventName == FName("Fire"))
    {
        SpawnProjectile(); // 정확한 프레임에 투사체 생성
    }
    else if (EventName == FName("Charge"))
    {
        ChargeLevel += 1; // 차지 단계 설정
    }
}
```

GAS는 Task 기반으로 흐름을 제어하여, 복잡한 몽타주 섹션 점프나 미세한 프레임 단위 제어 시 코드가 비대해질 수 있음

AnimNotify가 곧 트리거가 되는 구조이므로, 애니메이터/기획자가 몽타주 트랙에서 타이밍을 직접 수정하기 매우 용이함

Multiplayer Action Sync 멀티플레이 환경의 시간 동기화 한계를 극복한 Montage PlayRate 제어 시스템

배경 싱글에서는 Global Time Dilation을 조절해 타격감(Hit Stop)을 주지만, 멀티플레이에서는 모든 유저의 시간이 느려지는 문제 발생
Host(Listen Server)와 Client간의 RTT로 인해, 피격 시점과 연출 재생 시점의 불일치 발생

해결 월드 전체가 아닌 타격 대상과 공격자의 AnimMontage Play Rate만 일시적으로 줄였다가 복구하는 방식 적용
무기 타입에 따라 Hit Stop 시간과 카메라 쉐이크 강도를 다르게 적용하여 "무기의 무게감" 구현

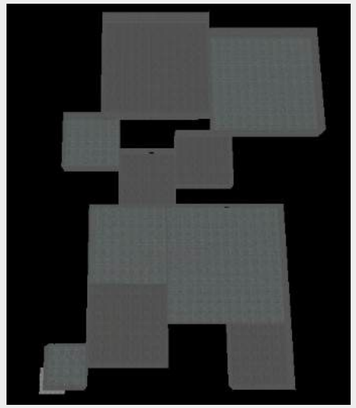
멀티 서버에서 데미지 판정 성공시, 공격자 클라이언트에게 Client RPC 전송
공격자에게만 Client RPC를 전송하여 대역폭 낭비 최소화 / 타인의 공격의 효과를 방지해 시각적 피로도 감소

[Montage 재생]

[재생 속도 조절]

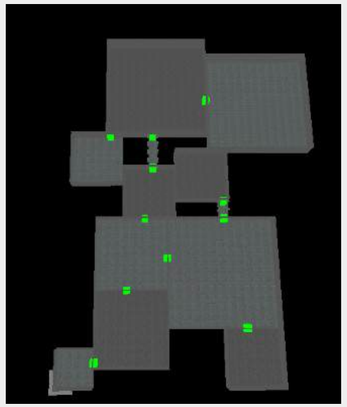
[재생 속도 복구]

Procedural Generation 절차적 던전 생성 알고리즘



Recursive Expansion

초기 시드(Seed) 방을 중심으로,
새로운 방을 기존 방의 상하좌우 면에
무작위로 부착해 나가는
확장형 배치 방식 적용



Graph Connectivity

배치된 방을 Node로 간주
거리 기반의 가중치를 계산하여 그래프화
Kruskal을 통해 모든 방이 연결된 MST
"고립된 방" 문제 원천 차단

Content Building

방 크기에 적합한 방 인테리어 생성
그리드 기반 배치로 그래프에 적용
무작위로 방 타입 및 몬스터 할당

생성한 방 그래프는 A*및 AI Agent에 활용

로그라이크의 무작위성은 '불확실성'을 의미하지만,
시스템 엔지니어에게는 '통제 가능한 변수'여야 합니다.
본 시스템은 그래프 이론(MST)을 적용하여 100%의 연결성을
수학적으로 보장하며, 생성된 데이터를 즉시 A* 그리드로 재활용하는
파이프라인을 통해 런타임 성능 부하를 완벽하게 해결했습니다

Data Pipeline ID 기반의 Zero-Allocation 동기화

아이디어 장비나 스킬등을 생성 파괴하는 것은 비용이 매우큼
서버와 통신할 때 특정 객체를 나타낼 ID가 필요함

판단 기존 객체는 남기고, ID만을 이용해 요소를 갱신하는 방식 채택

성과

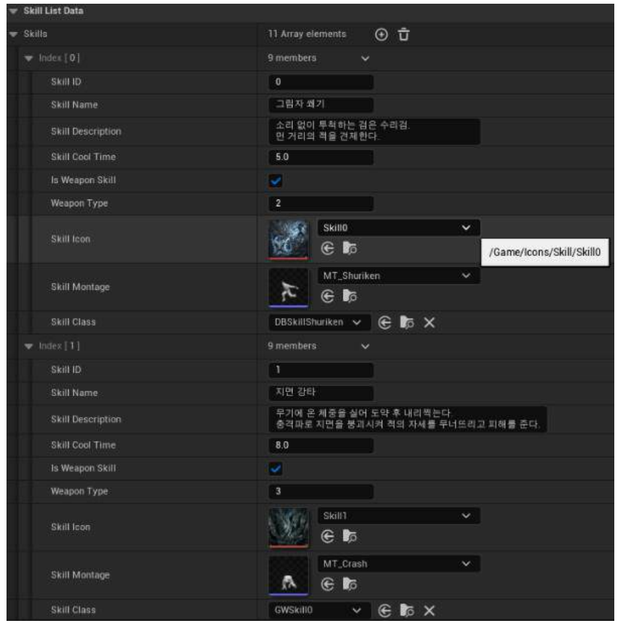
C++ 템플릿 기반의 범용 파서로 자동화된 파이프라인을 구축

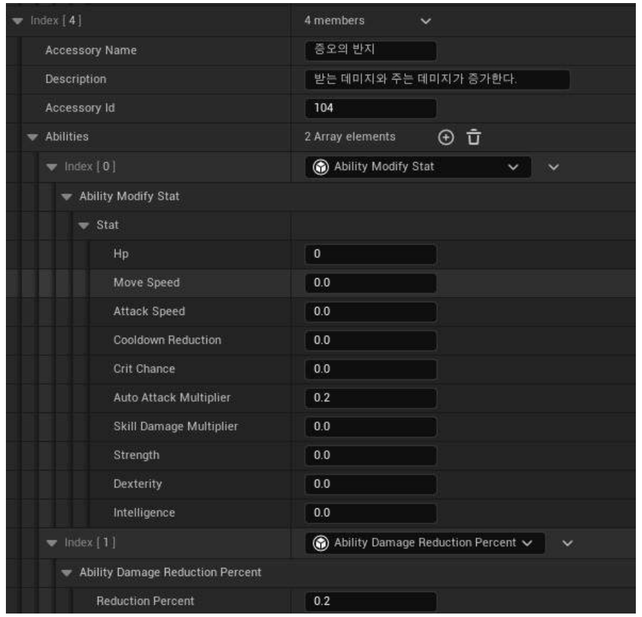
Google Sheet를 이용해 기획 생산성 확보

ID만 이용 서버와의 네트워크 대역폭 최적화

GC 스파이크 방지 및 Zero-Allocation 방식을 통해
메모리 최적화 달성

전투 밸런스를 기획자가 수정하고 확인할 수 있는 환경 제공





[Data Asset기반 데이터 관리]

monsterId	monsterType	monsterName	hp	speed	attack	attackSpeed	attackRange	staggerGage	exp	mesh
0	0	스켈레톤	450	185	30	1	300	10	10	/Game/Blueprints/Monster/BP_TestMonster
1	0	저주받은 고사지	300	400	75	1	250	50	8	/Game/Blueprints/Monster/BP_Stickman
2	0	거미	500	250	40	0.7	200	100	10	/Game/Blueprints/Monster/BP_Spider
3	1	언데드	900	100	50	1.5	170	100	30	/Game/Blueprints/Monster/BP_ReviveZombie
4	0	스켈레톤 석궁병	400	150	30	1	2000	10	10	ne/Blueprints/Monster/BP_CrossBowSkeleton
5	1	천번찔린 언데드	10	200	60	0.8	170	100	25	ame/Blueprints/Monster/BP_HitCountZombie

weaponId	weaponType	weaponName	rarity	AttackPower	StaggerPower	str	dex	int	mesh
0	1	일반 숏소드	0	10	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/Sta
1	1	고급 숏소드	1	12	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/Sta
2	1	레어 숏소드	2	15	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/Sta
3	1	레전드 숏소드	3	20	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/Sta
4	1	일반 한손검	0	13	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/Sta
5	1	고급 한손검	1	16	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/Sta
6	1	레어 한손검	2	18	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/Sta
7	1	레전드 한손검	3	22	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/Sta

[CSV 기반 데이터 관리]

화재 대비 시뮬레이터

기간 : 25.09.29 ~ 25.11.03

두 명의 플레이어가 협력하여
화재 상황에서 안전하게 대피하는 교육용 시뮬레이션 게임
화재 발생 시 상황 직접 체험하며 학습할 수 있도록 설계

역할

1. 옥트리 : 동적 공간 분할
2. 플레이어 동기화
3. 네트워크 최적화

참여 인원

총 3명

영상 링크

https://drive.google.com/file/d/1sPOko1ewEnb0zkySgrAd_x8OIPogF_6x/view

깃허브 링크

https://github.com/rouge-like/Horizontal

노션

https://www.notion.so/2a76884456e5808787b9dc9a69e5b483

주요 성과

알고리즘 및 데이터 구조 최적화
네트워크 동기화

Spatial Partitioning 화재 확산 구현을 위한 공간 분할

"화재 확산을 구현하기 위해 주변에 확산 할 수 있는 객체인지 판단 후 확산하는 기능 필요"

개별 액터 체크

시도 각 불 객체가 주변의 연소 가능 물체를 직접 감지

한계 불이 번져 객체가 수백 개로 늘어나자, 매 프레임
수백 번의 충돌 검사가 발생하여 기하급수적인 연산 부하 발생

판단 개별 액터가 아닌, 화재 구역 전체를 관리하는
매니저(Manager) 형태의 시스템이 필요

균일 그리드

시도 공간을 일정 크기의 셀(Cell)로 나누어 관리하는
3차원 배열(Grid) 방식 도입

한계 건물 전체를 커버하려니 $O(N^3)$의 반복문이 필요
-> 크기가 커질 수록 기하급수적인 연산 부하 발생

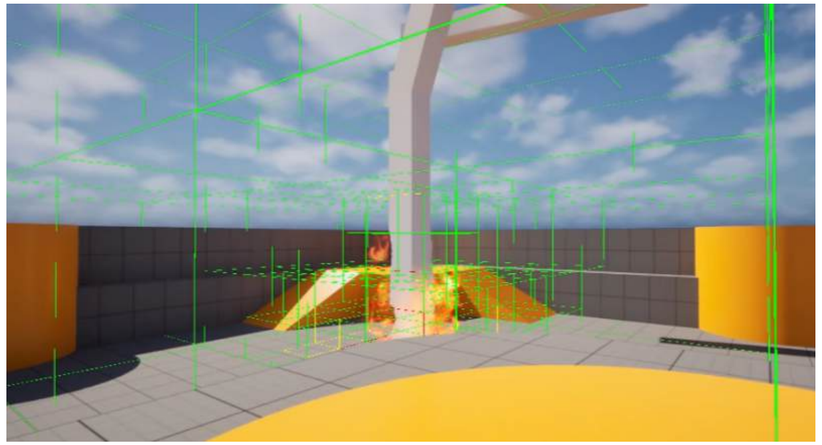
판단 화재가 없는 빈 공간까지 모두 순회해야 하므로 비효율적,
빈공간을 최소화 하는 자료구조가 필요

Octree : Dynamic Subdivision

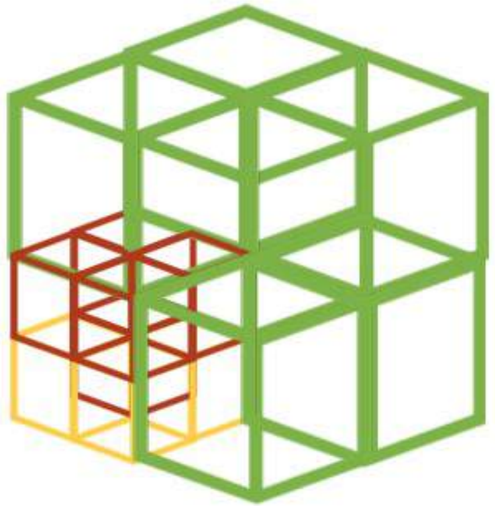
확산 연산의 기하급수적 증가와 메모리 비효율을 해결

아이디어 유니티/언리얼 등 상용 엔진이 물리 충돌 판정할 때, 모든 공간을 검사하지 않고 객체가 있는 곳만 쪼개서 검사한다는 점에 착안

판단 화재는 특정 구역에 집중되고 나머지 공간은 비어있는 희소한 데이터 분포를 가짐 따라서 고정된 그리드보다 유동적인 옥트리가 적합하다고 판단



```
// 3차원 공간을 8개의 하위 셀로 분할 (Subdivision)
for (int32 X = 0; X < 2; ++X) {
    for (int32 Y = 0; Y < 2; ++Y) {
        for (int32 Z = 0; Z < 2; ++Z) {
            // 자식 노드의 중심 좌표 계산
            const FVector ChildCenter = ParentCenter + FVector(OffsetX, OffsetY, OffsetZ);
            // 새 셀 생성 및 인덱스 연결 (Create & Link)
            const int32 ChildIndex = CreateCell(ChildCenter, ChildExtent, ...);
            Cells[CellIndex].ChildIndices[ChildCounter++] = ChildIndex;
        }
    }
}
```



[초록] 비활성 영역
[노랑] 관심 영역
[빨강] 발화 영역

Octree
x, y, z 축을 각각 2등분하여 총 8개의 자식 노드를 생성 '관심 구역'만 재귀적으로 분할하여 메모리와 연산 효율 최적화 불필요한 빈 공간 순회를 제거하여 프레임 안정화 성공

Octree : Merge & Optimization 옥트리 탐색 깊이와 메모리 사용량 최소화

Why

소화가 되어도 작은 노드들이 남아있으면, 화재가 없는데도 깊은 트리 탐색 비용이 발생함
이를 하나의 큰 노드로 합쳐, 다음 화재 판정 시 O(1)로 빠르게 통과하도록 최적화

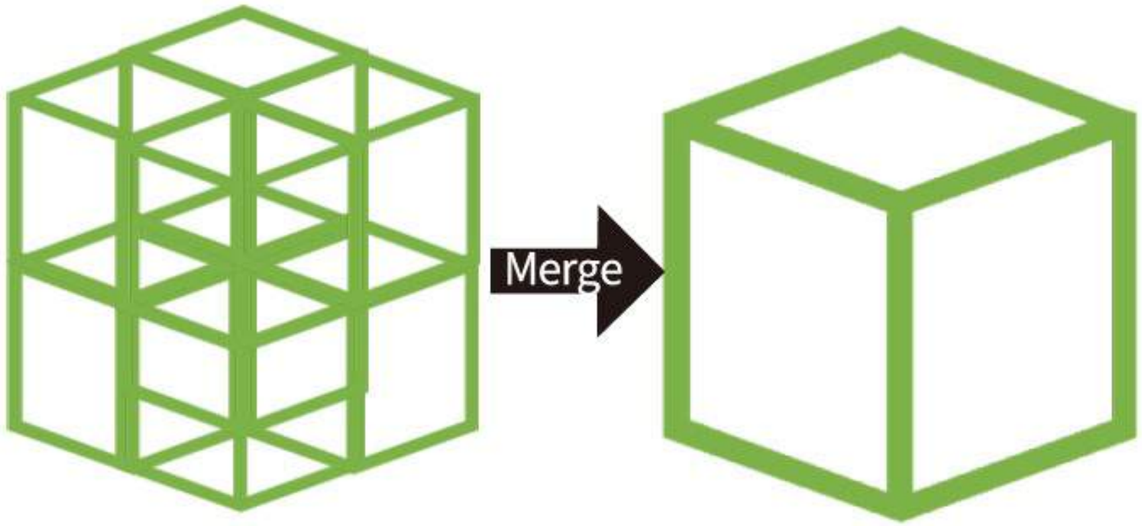
How

매 프레임 병합을 검사하면 오히려 오버헤드가 발생
따라서 1초 주기(Throttling) 지연 병합을 도입하여 성능 균형 유지

```
// 하위 노드 상태 검사 및 병합 판단 (Bottom-up Check)
bool bAllChildrenExtinguishedOrDormant = true;

for (int32 i = 0; i < 8; i++) {
    // 재귀적으로 자식 노드 상태 확인
    FFireCellTickResult ChildResult = ProcessCollapseRecursive(ChildIndex, ...);

    const FFireCell& ChildCell = Cells[ChildIndex];
    if (ChildCell.State != EFireCellState::Dormant && ChildCell.State != EFireCellState::Extinguished)
    {
        bAllChildrenExtinguishedOrDormant = false;
    }
}

// 모든 자식(Leaf)이 안전(Extinguished/Dormant)하다면 병합 수행
if (bAllChildrenLeaf && bAllChildrenExtinguishedOrDormant)
{
    CollapseCell(CellIndex); // 하위 노드 제거 및 메모리 풀 반환
    Cells[CellIndex].State = EFireCellState::Extinguished; // 부모 상태 갱신
}
```

Bottom-up Merge

재귀적으로 모든 자식 노드가 Leaf 상태이고, 안전한지를 검증하여 병합
병합 성공시 8개의 노드가 1개로 줄어, 연산량을 획기적으로 감소시킴



Troubleshooting : Invalid Reference

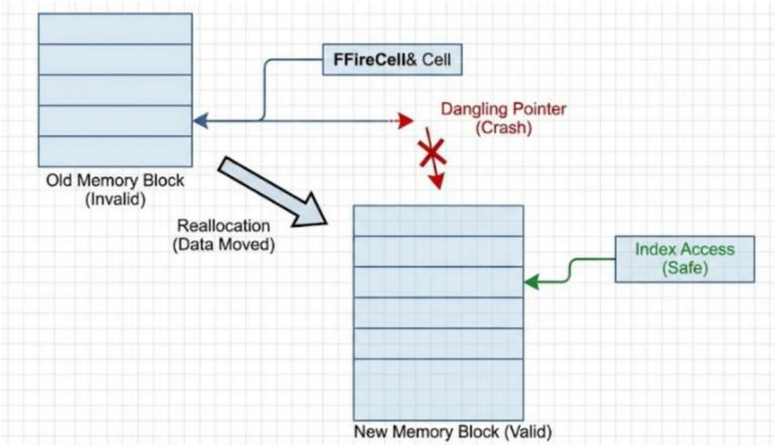
동적 배열 재할당(Reallocation)으로 인한 포인터 무효화 문제 해결

문제 간헐적으로 Octree가 분할하지 않는 오류 확인

추적 Debug Log와 Breakpoint를 설정하여 옥트리 배열의 상태 실시간 모니터링
CreateCell 함수가 호출되는 순간, 기존에 잘 참조하던 부모 노드의 주소값이
유효하지 않은 값으로 변하는 것을 포착

분석 언리얼 TArray는 요소가 추가되어 Capacity를 초과하면,
더 큰 메모리 공간을 찾아 Reallocation를 감행함
이때 참조를 잡고 있던 변수는, 이전 주소를 계속 가리키게 됨
결과적으로 Dangling Pointer 접근으로 인한 데이터 오염 발생

해결 주소가 아닌, 변하지 않는 Unique ID(Index)를 사용
참조가 필요할 때마다 Cells[Index]로 접근하여,
메모리 위치가 바뀌어도 항상 올바른 데이터를 가져오도록 구조 변경



[Before]

```
FFireCell& Cell = Cells[CellIndex];

// 여기서 Reallocation 발생
int32 ChildIndex = CreateCell(...);

// 접근 위반 발생
Cells.ChildIndices[ChildCounter++] = ChildIndex;
```

[After]

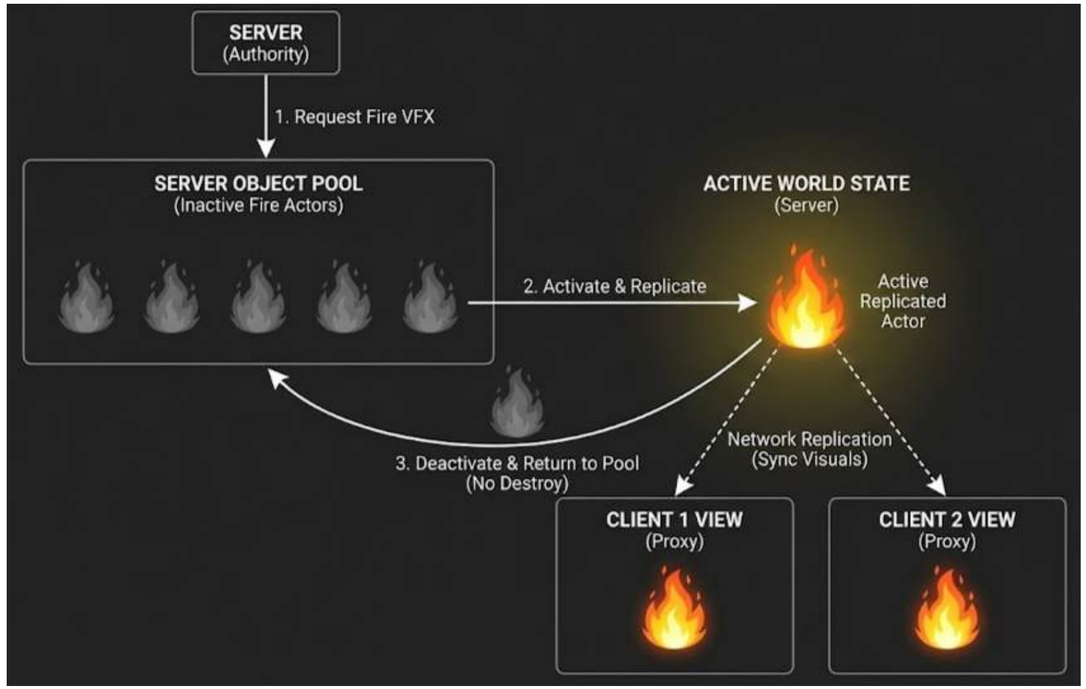
```
// 메모리 주소가 바뀌어도 Index는 유지됨
int32 ChildIndex = CreateCell(...);

// 사용 할 때 인덱스로 접근
Cells[CellIndex].ChildIndices[ChildCounter++] = ChildIndex;
```

Multiplayer Optimization : Replicated Object Pooling

네트워크 대역폭 낭비 및 프레임 드랍 방지

문제 불 이펙트와 같이 생성/파괴가 잦은 시스템으로 인해 프레임 드랍이 발생함
대량의 이펙트 동기화 시 발생하는 네트워크 대역폭 급증 문제 해결 필요

해결 Replicated Actor를 관리하는 Pooling 시스템을 구성해 생명주기를 관리함



[Pre-allocation] 초기 풀 생성

```
// 설정된 개수(InitialPoolSize)만큼 미리 생성
for (int32 Index = 0; Index < ActorsToCreate; ++Index)
{
    // 1. 액터 생성 (메모리 할당)
    AVFXActor* VFXActor = GetWorld()->SpawnActor<AVFXActor>(...);
    // 2. 비활성화 상태로 대기 (Sleep)
    VFXActor->SetActorHiddenInGame(true); // 렌더링 끔
    VFXActor->SetActorTickEnabled(false); // 연산 끔

    Pool.VFXs.Add(VFXActor);
}
```

[Spawn] 활성화 및 동기화

```
for (int32 Index = 0; Index < Pool.VFXs.Num(); ++Index)
{
    AVFXActor* VFXActor = Pool.VFXs[Index];

    if (VFXActor->IsHidden())
    {
        ..상태 초기화 (위치, 회전, 스케일, Tick활성화 등)
        VFXActor->StartVFX();

        return VFXActor;
    }
}
```

[Despawn] 비활성화 및 반환

```
if (VFXActors.Contains(VFX->GetVFXName()))
{
    ...비활성화 (Hidden, Tick비활성화 등)
    VFX->StopVFX();
}
```

헬다이버즈

기간 : 25.09.01 ~ 25.09.24

헬다이버즈2의 모작 프로젝트
컴포넌트 기반의 모듈형 설계로 의존성을 최소화한 3인 협업 TPS 프로젝트

역할

1. 모듈 아키텍쳐
2. 전투 시스템
3. 미션 프레임워크

참여 인원

총 3명

영상 링크

https://drive.google.com/file/d/1PjTi9cVO1Apd7aCgXNdblES7R-6p5KMN/view?usp=drive_link

깃허브 링크

https://github.com/rouge-like/ForSuperDemocracy

노션

https://www.notion.so/Helldivers-2a86884456e5801da148f2eeb4b6280c

주요 성과

디커플링 시스템 - 팀원간 코드 충돌을 0에 가깝게 줄임
컴포넌트들의 높은 재사용성 확보
Data-Driven 디자인을 통해 데이터 수정의 유연성 확보

Modular Architecture & Component Pattern 의존성 최소화를 통한 협업 병목 해결

기존 협업

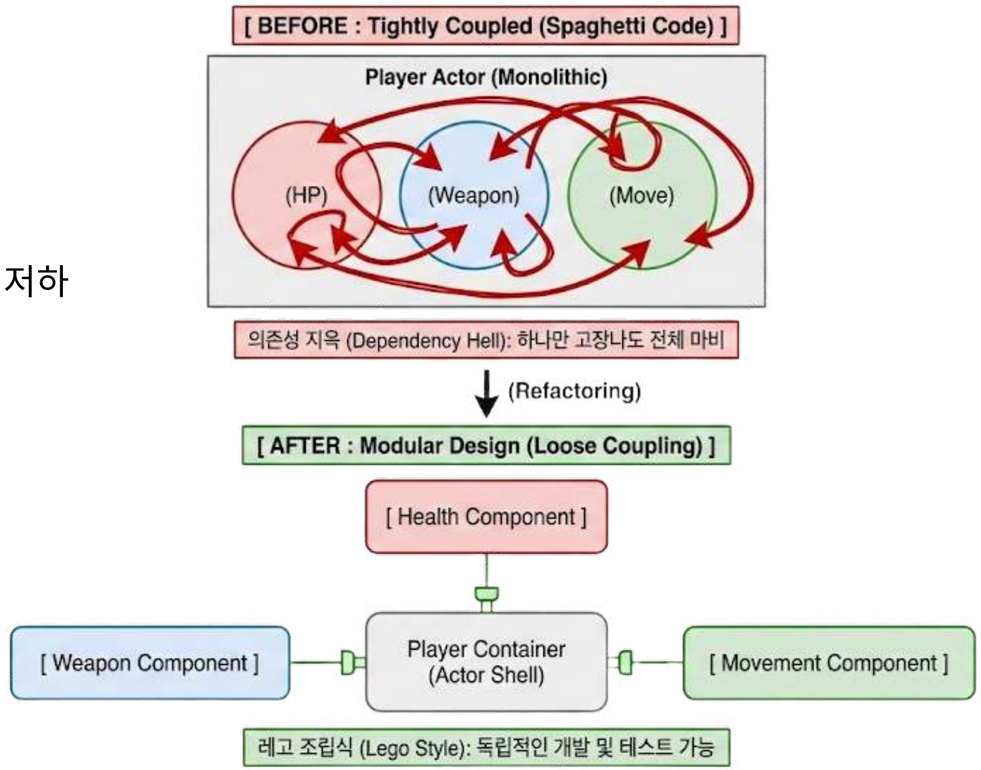
배경 기존에는 한 액터 안에 여러 로직이 뒤섞여 있었음

한계 팀원의 코드가 완성되지 않으면 내 코드를 테스트할 수 없는
상호 의존성(Dependency) 문제 발생
누군가 코드를 고치면 내 기능이 함께 망가지는 현상으로 협업 효율 저하

모듈화 분업

시도 액터는 Container 역할만 수행, 기능 단위로 철저히 분리

효과 팀원 간 작업 충돌 없이 병렬 개발 가능
(예: A는 체력 로직만, B는 무기 로직만 수정해도 서로 영향 없음)

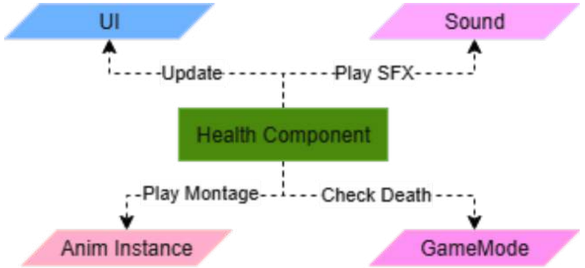


"언리얼 엔진의 설계 철학인 Actor와 Component 구조를 채택했는지 몸소 체감"

Modular Architecture & Event-Driven 협업 효율을 위한 디커플링 설계

문제 강한 결합으로 인해 코드 수정 시 연쇄적인 오류 발생 / 협업 대기 시간 증가

해결 Observer Pattern 적용 / 기능과 표현을 완전히 분리



[Health Component 느슨한 결합]

```
void UHealthComponent::BeginPlay()
{
    Super::BeginPlay();

    if (AActor* Owner = GetOwner())
    {
        // Owner가 Player든 Monster든 상관없이 바인딩 (다형성 활용)
        Owner->OnTakePointDamage.AddDynamic(this, &UHealthComponent::HandlePointDamage);
        Owner->OnTakeRadialDamage.AddDynamic(this, &UHealthComponent::HandleRadialDamage);
    }
}
```

범용적 이벤트 바인딩

Owner가 누군지 가리지 않고
엔진 표준 데미지 이벤트를 Hook하여 동작

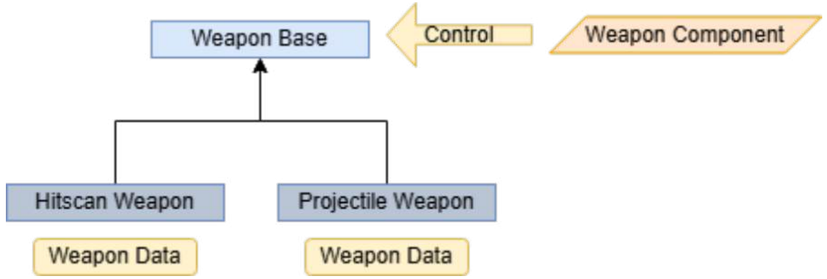
```
void UHealthComponent::ApplyDamageInternal(float Damage, ...)
{
    CurrentHealth = FMath::Clamp(CurrentHealth - Damage, 0.f, MaxHealth);

    // 결과 방송 : UI, Sound, VFX 등이 이 이벤트를 구독하여 반응
    OnDamaged.Broadcast(...);
    OnHealthChanged.Broadcast(...);

    // 사망 처리: 스스로 사망 상태를 판단하고 방송
    if (CurrentHealth <= 0.f && bCanDie)
    {
        OnDeath.Broadcast(GetOwner()); // 외부(GameMode 등)에 사망 알림
        OnRagdoll(); // 물리 시뮬레이션, 래그돌 변환
    }
}
```

의존성 없는 상태 전파

결과를 Delegate로 방송하여
UI, 애니메이션 등이 컴포넌트들을 직접 참조하지 않도록 의존성 제거

Polymorphic & Data-Driven 상속을 통한 생산성 향상과 기획 효율을 위한 데이터 기반 설계

[무기의 다형성]

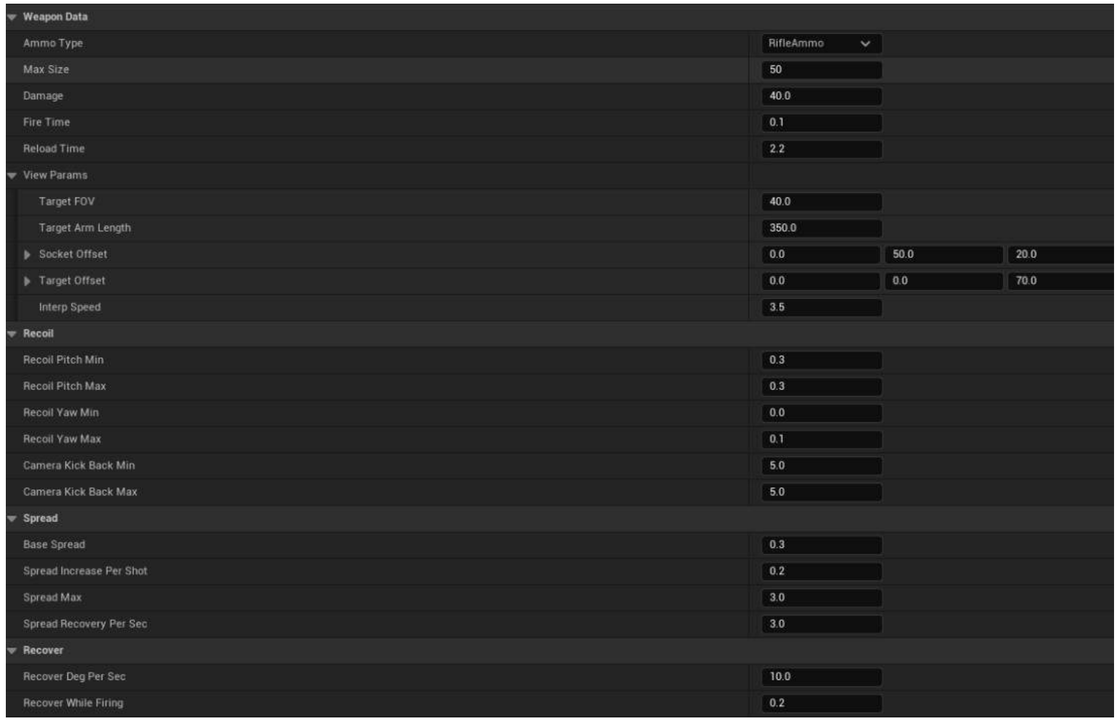



```
void AWeaponBase::FireOnce()
{
    // 탄약 소모, 반동적용 등 공통 로직 수행
    OnFired.Broadcast(this);
}
void AHitscanWeapon::FireOnce()
{
    Super::FireOnce();
    GetWorld()->LineTraceSingleByChannel(...);
}
void AProjectileWeapon::FireOnce()
{
    Super::FireOnce();
    GetWorld()->SpawnActor<AActor>(...);
}
```

다형성을 활용한 무기 확장

탄약 소모, 재장전, 반동 등 공통 로직은 부모에서 한번
FireOnce를 가상함수로 설계하여, 자식에서 직접 구체화

[무기의 DataAsset]



Weapon Data			
Ammo Type	RifleAmmo		
Max Size	50		
Damage	40.0		
Fire Time	0.1		
Reload Time	2.2		
View Params			
Target FOV	40.0		
Target Arm Length	350.0		
Socket Offset	0.0	50.0	20.0
Target Offset	0.0	0.0	70.0
Interp Speed	3.5		
Recoil			
Recoil Pitch Min	0.3		
Recoil Pitch Max	0.3		
Recoil Yaw Min	0.0		
Recoil Yaw Max	0.1		
Camera Kick Back Min	5.0		
Camera Kick Back Max	5.0		
Spread			
Base Spread	0.3		
Spread Increase Per Shot	0.2		
Spread Max	3.0		
Spread Recovery Per Sec	3.0		
Recover			
Recover Deg Per Sec	10.0		
Recover While Firing	0.2		

데이터 주도적 설계

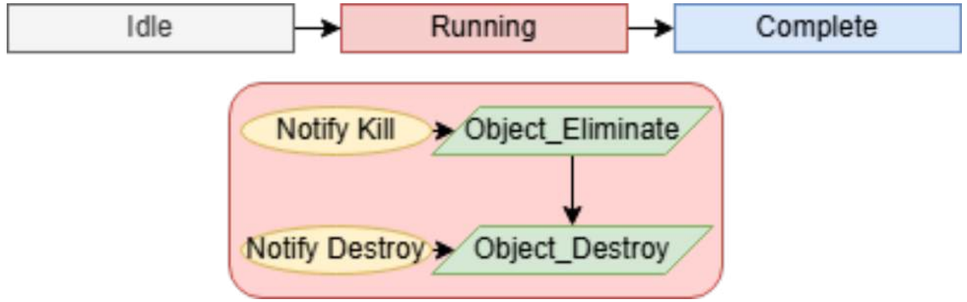
데미지, 탄약 수 등 데이터를 DataAsset으로 관리
반동, 탄퍼짐, 카메라 세팅 등 슈팅 디테일도 함께 관리
기획자가 코드 재컴파일 없이 에디터에서 즉시 밸런싱 가능

Mission Flow & Objective Polymorphism

확장 가능한 상태 머신 설계를 통한 미션 로직의 모듈화

문제 단순 if-else문으로 다양한 미션(처치, 생존 등)을 관리할 경우,
코드 결합도가 높아져 새로운 미션 추가 시 유지보수가 어려움

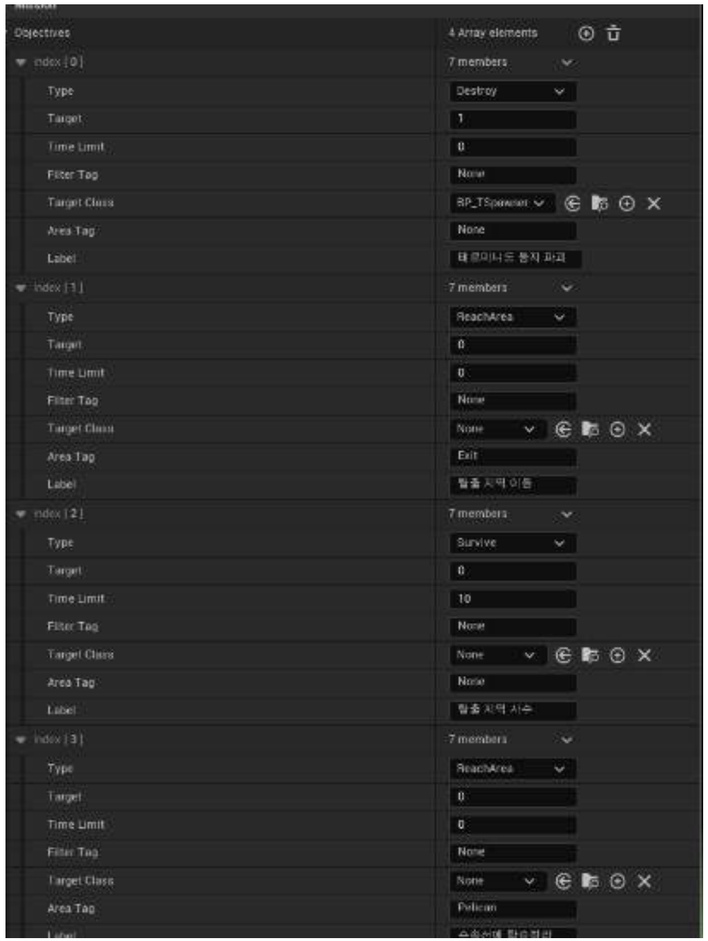
해결 미션의 생명주기 상태별로 분리하여 로직 간섭 방지
구체적인 목표 로직을 캡슐화하여, 기존 코드 수정 없이 새로운 미션 타입 확장 가능



FSM 기반 Flow 제어

현재 미션 상태에 따라 받는 이벤트 제어(Kill, Survive, Destroy 등)
상태 변경 시 델리게이트로 방송하는 옵저버 패턴 적용
UI가 데이터를 직접 참조하지 않는 느슨한 결합 구현
미션은 DataAsset으로 관리

[Mission System Data]



타이니 서바이버

상용 엔진의 네트워크 모듈에 의존하지 않고, 게임 서버의 핵심인 '대규모 동시성 제어'와 '패킷 처리 파이프라인'의 원리를 밑바닥 부터 규명하기 위한 개인 프로젝트

특징

네트워크(IO)와 게임 로직을 분리

Zone 기반 시야 관리로 $O(N^2)$ 브로드캐스트 비용을 O(1)로 최적화

성과

스마일 게이트 퓨처랩 INDEGO 2024 출품

영상 링크

https://www.youtube.com/watch?v=FlYFEVaKxdc

깃허브 링크

https://github.com/rouge-like/Server

노션

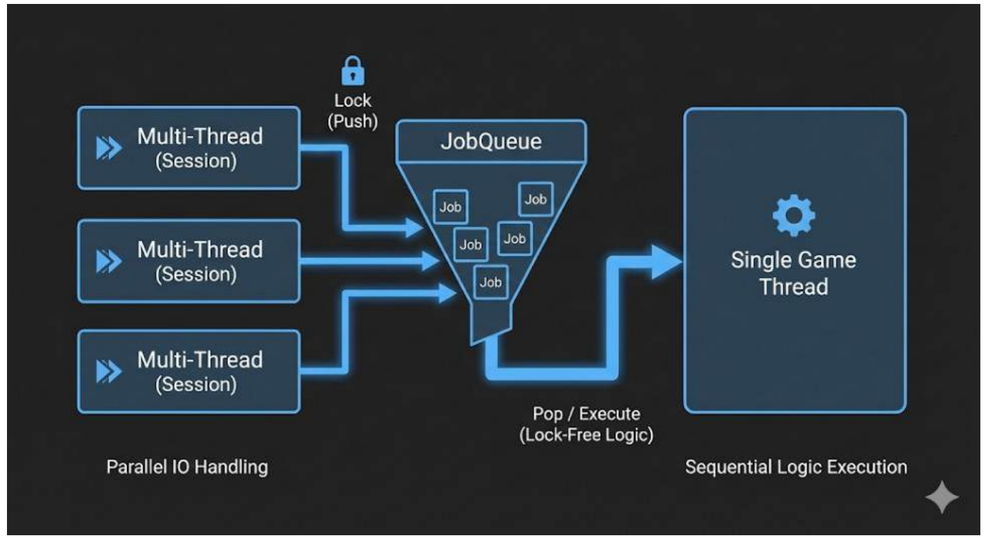
https://www.notion.so/Tiny-Survivor-Server-28a6884456e580abbeebff66f1b87a9e

Concurrency Architecture

JobQueue를 활용한 Lock-Free 설계로 데드락 원천 차단 및 컨텍스트 스위칭 최소화

문제 멀티스레드 서버에서 공유자원에 잦은 Lock을 걸면 Deadlock 위험과 Context Switching 비용 발생

해결 JobQueue & JobSerializer



Single Threaded Execution	네트워크 수신은 멀티스레드에서 처리하되 실제 로직 처리는 작업을 객체화하여 큐에 넣고 Game Thread가 이를 독점적으로 순차 처리
성과	**데이터 무결성** 데이터 충돌 없이 안전하게 처리 Race Condition 0% **로직간 간섭 최소화** 네트워크 I/O와 DB 작업은 병렬로 처리되지만, 핵심 게임 로직은 순차 처리되어 데이터 간섭 없음 **Lock 최소화** 공유 자원을 Lock 없이 사용 DeadLock 원천 차단 스위칭 비용 최소화
스케쥴링	우선순위 큐 구조로 Job에 시간을 설정하여, 작업 스케쥴링이 가능

Network I/O & Serialization

Protobuf 도입으로 직렬화 비용 최소화 및 TCP 패킷 처리

```
// [RecvBuffer] TCP 스트림 처리를 위한 슬라이딩 윈도우 구현
public void Clean()
{
    int dataSize = DataSize; // 남은 유효 데이터 크기
    if (dataSize == 0)
    {
        // 데이터가 없으면 커서 초기화 (복사 비용 0)
        _readPos = _writePos = 0;
    }
    else
    {
        // 남은 데이터만 버퍼 앞쪽으로 이동 (Memory Copy)
        Array.Copy(_buffer.Array, _buffer.Offset + _readPos, _buffer.Array, _buffer.Offset, dataSize);
        _readPos = 0;
        _writePos = dataSize;
    }
}
```

TCP Stream 처리

TCP 특성상 패킷이 뭉치거나 잘려서 올 수 있음
슬라이딩 버퍼 구조 RecvBuffer를 통해 해결
구글 Protobuf 도입으로 JSON 대비 Payload 크기 최소화 및 직렬화 비용 감소

```
public static void C_MoveHandler(PacketSession session, IMessage packet)
{
    // 1. Protobuf 메시지 캐스팅 (직렬화 해제된 객체 사용)
    C_Move movePacket = packet as C_Move;
    ClientSession clientSession = session as ClientSession;

    // 2. 유효성 검사 (Player/Room이 존재하지 않으면 패킷 무시)
    Player myPlayer = clientSession.MyPlayer;
    Room room = myPlayer?.Room;

    if (room == null) return;

    // 3. 로직 처리를 GameThread(JobQueue)로 위임 (Context Switching 최소화)
    // 여기서 즉시 실행하지 않고 큐에 넣음으로써 Lock을 걸 필요가 없음
    room.Push(room.HandleMove, myPlayer, movePacket);
}
```

조립된 패킷 로직 큐에 넣기

Protobuf가 생성한 클래스로 안전하게 형변환하여 사용
네트워크를 처리하는 스레드가 게임 로직을 직접 건드리지 않고,
일감(Job)만 던져주고 즉시 복귀함

Spatial Partitioning

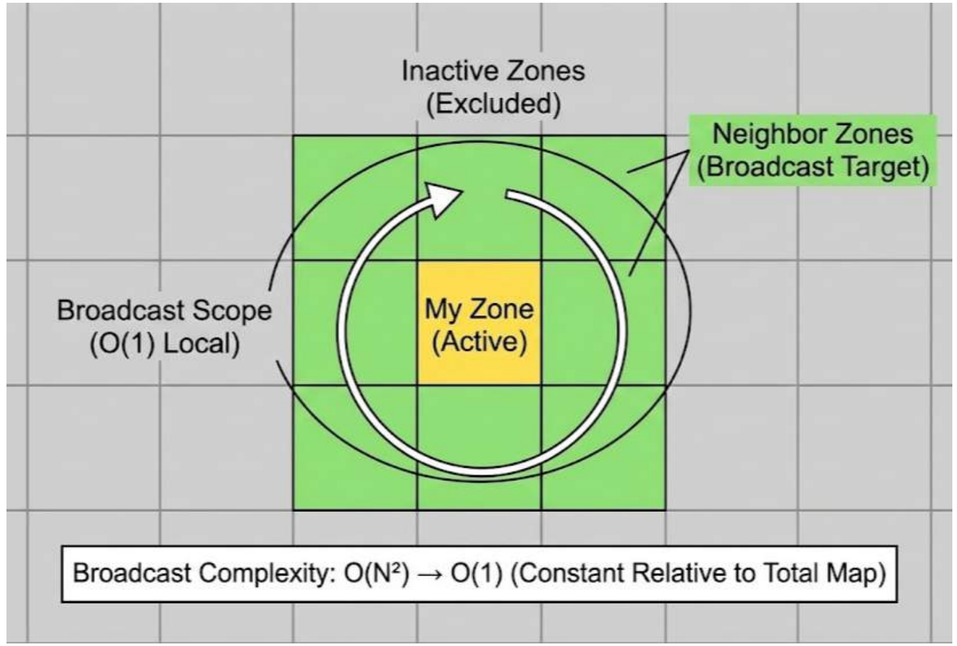
공간 분할과 HashSet을 활용한 $O(N^2)$브로드캐스트 문제 해결

Zone-Based Partitioning

문제 MMO에서 모든 유저가 서로의 정보를 주고받으면 패킷량은$O(N^2)$로 폭증

해결 맵을 일정 크기의 격자(Zone)로 분할 내 캐릭터가 위치한 Zone과 인접한 8개 Zone의 유저에게만 패킷 전송

결과 전체 접속자가 아무리 많아도, 내 주변 유저 수만큼만 연산하므로 O(1) 복잡도 유지 (전체 맵 대비)



Hash Set

배경 시야 내에 들어온 오브젝트 목록을 관리하고, 이동할 때마다 갱신해야 함

문제 특정 유저가 시야에서 나갔는지 확인할 때 탐색 비용 O(N)발생

해결 HashSet<T> 도입
데이터 유무 확인, 추가, 삭제가 평균 O(1)로 수행됨
빈번하게 시야에 들어오고 나가는 MMO 환경에 갱신 부하를 최소화

Performance Profiling & Tuning

프로파일링 데이터를 기반으로 병목 구간을 탐지하고 서버 수용량 증대

CPU 병목

문제 A*기반 FindPath 함수가 프레임 드랍을 유발

해결 무거운 A* 대신 경량화된 추적 알고리즘으로 전환

결과 연산 비용을 획기적으로 줄여 서버 수용량 증대

10.92% -> 0.92%

[Before]

Server.Contents.Monster.Update()	9860(11.76%)
Server.Contents.Monster.UpdateMoving()	9583(11.42%)
Server.Contents.Map.FindPath(Server.Cont...	9158(10.92%)
Server.Contents.Monster.BroadcaseMove()	381(0.45%)
Server.Contents.Map.MoveObject(Server.C...	17(0.02%)
Server.Contents.GameObject.get_CellPos()	9(0.01%)
Server.Contents.GameObject.GetFrontCell...	4(0.00%)

[After]

Server.Contents.Monster.Update()	1115(0.92%)

논리적 누수

문제 게임이 지속될 경우 CPU 부하가 점차 증가하는 현상 발견

원인 Player가 사망하여 객체가 제거돼도 VisionCube가 계속 남아 있어 Update() 수행

해결 Player 파괴 시점에 리소스 파괴 작업을 명시적으로 예약

결과 VisionCube의 점유율이 상시 0.1%대를 유지

[초기 점유율]

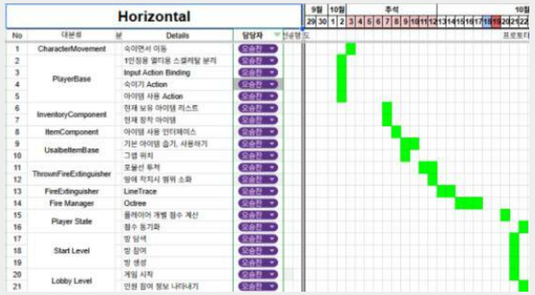
Server.Contents.Player.Update()	46(0.20%)
Server.Contents.Object.Dark.Update()	42(0.18%)
Server.Contents.VisionCube.Update()	37(0.16%)
Server.Contents.Object.Lightning.Update()	23(0.10%)
Server.Contents.Item.Update()	15(0.06%)
Server.Contents.Projectile.Update()	15(0.06%)
Server.Contents.Area.Update()	11(0.05%)

[10분 후 점유율]

Server.Contents.Job.Excute()	2907(12.40%)
Server.Contents.VisionCube.Update()	1623(6.92%)
Server.Contents.Room.SpawnMonster()	603(2.57%)
Server.Contents.Object.Sword.Update()	261(1.11%)
Server.Contents.Monster.Update()	179(0.76%)
Server.Contents.Object.Dark.Update()	60(0.26%)
Server.Contents.Player.Update()	49(0.21%)

협업 관리

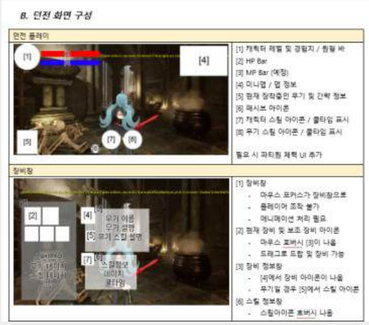
일정관리 개발 로드맵의 가시화



주 단위 스프린트 계획을 간트 차트로
시각화하여 팀원 간 업무 진행 상황 투명하게 공유

병목 구간을 사전에 파악하고,
일정 지연 리스크를 최소화

기획 명세 오해 없는 기획 의도 전달



추상적인 텍스트 기획서 대신,
도식화된 순환도와 UI 명세서를 통해
팀원 간의 모델 일치화
구현 단계에서의 재작업 비용 0% 목표

데이터 규격화 코드와 기획의 연결 고리 설계

monsterId	monsterType	monsterName	hp	speed	attack	attackSpeed	attackRange	staggerGage	exp	mesh
0	0	스플래쉬	350	350	30	1	300	10	10	/Game/Blueprints/Monster/BP_TestMonster.BP_TestMonster_C
1	0	저주받은 고시지	150	400	75	1	250	50	8	/Game/Blueprints/Monster/BP_Stickman.BP_Stickman_C
2	0	거미	400	250	40	0.7	200	100	10	/Game/Blueprints/Monster/BP_Spider.BP_Spider_C
3	1	언데드	800	150	50	1.5	170	100	30	/Game/Blueprints/Monster/BP_ReviveZombie.BP_ReviveZombie_C
4	0	스켈레톤 석궁병	300	350	30	1	2000	10	10	ne/Blueprints/Monster/BP_CrossBowSkeleton.BP_CrossBowSkelet
5	1	한번쯤은 언데드	10	300	60	0.8	170	100	25	\ame/Blueprints/Monster/BP_HitCountZombie.BP_HitCountZombie

weaponId	weaponType	weaponName	rarity	AttackPower	StaggerPower	str	dex	int	mesh
0	1	일반 숏소드	0	10	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/StaticMeshes/ShortSword.ShortSword
1	1	고급 숏소드	1	12	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/StaticMeshes/ShortSword.ShortSword
2	1	레어 숏소드	2	15	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/StaticMeshes/ShortSword.ShortSword
3	1	레전드 숏소드	3	20	2	0.7	0.3	0	/Game/Assets/Weapons/OneHandedSword/ShortSword/StaticMeshes/ShortSword.ShortSword
4	1	일반 한손검	0	13	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/StaticMeshes/ArmingSword.ArmingSwor
5	1	고급 한손검	1	16	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/StaticMeshes/ArmingSword.ArmingSwor
6	1	레어 한손검	2	18	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/StaticMeshes/ArmingSword.ArmingSwor
7	1	레전드 한손검	3	22	2	0.5	0.5	0	/Game/Assets/Weapons/OneHandedSword/ArmingSword/StaticMeshes/ArmingSword.ArmingSwor

캐릭터 스탯, 아이템 ID 등 핵심 데이터를
엑셀로 규격화하여 공유

데이터가 코드의 Struct나 DataAsset으로
직관적으로 1:1 매핑되도록 구조 설계